מבדק פיתוח · Full Stack

סיכום הפרויקט

מערכת לניהול פניות

מערכת שמאפשרת לצוות לנהל פניות שמגיעות מארגונים: למצוא, לסנן, לעדכן סטטוס ולעקוב אחרי כל שינוי – גם כשכמה אנשים עובדים עליה בו־זמנית.

Angular 22 · .NET 10 · SQL Server · EF Core · Angular Material · GitHub Actions

471	43	100,000	M0–M9
בדיקות אוטומטיות	החלטות מתועדות	פניות בנתוני הדוגמה	עשרה שלבי עבודה

מבוסס על PROJECT_SUMMARY.md במאגר requests-management. הפירוט המלא נמצא במסמכים שהמסמך מפנה אליהם.

תוכן העניינים

1	**על הפרויקט**	מה המערכת עושה
2	**הארכיטקטורה במבט אחד**	החלקים הגדולים
3	**מודל הנתונים ומעברי הסטטוס**	מה נשמר ואיך פנייה מתקדמת
4	**השרת**	איפה מתבצעת העבודה
5	**הלקוח**	מה המשתמש רואה
6	**החלטות מרכזיות**	למה נבנה כך
7	**מגבלות ידועות ובחירות מודעות**	מה נבחר להשאיר פשוט
8	**תהליך העבודה**	איך העבודה התנהלה ואומתה
9	**מבנה התיקיות**	איפה כל דבר נמצא
10	**מפת המסמכים**	לאן להמשיך לקרוא
11	**מילון מונחים**	הסבר קצר למונחים הטכניים

1 על הפרויקט

במילים פשוטות

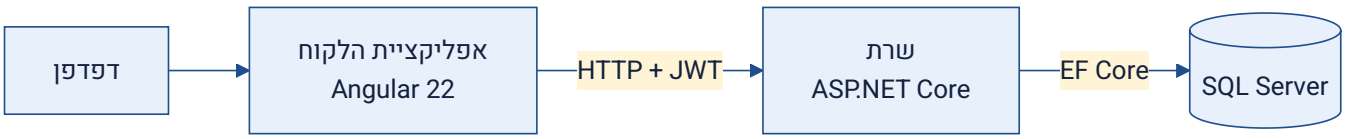
ארגונים שולחים פניות, וצוות מטפל בהן. המערכת מציגה את כל הפניות, מאפשרת לחפש ולסנן אותן במהירות, לשנות להן סטטוס ולראות מי שינה מה ומתי. היא נבנתה כך שתעבוד מהר גם עם מאה אלף פניות, ושלא "תדרוס" שינוי של אדם אחד כשאדם אחר עובד על אותה פנייה באותו רגע.

מערכת לניהול פניות ובקשות המתקבלות מארגונים, לעבודה של כמה משתמשים במקביל מול 000,100 פניות: איתור, סינון, חיפוש, מיון ודפדוף – כולם בצד השרת; עדכון סטטוס עם Optimistic Concurrency והיסטוריית שינויים; עדכון מרוכז (Bulk) של עד 100 פניות; ולוח בקרה עם נתונים מסכמים השמורים ב־Cache. הפתרון בנוי ב־.NET 10, Angular 22 ו־SQL Server, וכולו ב־Repository אחד: `be/` (Backend), `fe/` (Frontend) ו־`docs/` (תיעוד), עם ה־Issues וה־Milestones. המשימות מנוהלות ב־Project Board.

2 הארכיטקטורה במבט אחד

במילים פשוטות

יש שלושה חלקים: המסך שהמשתמש רואה בדפדפן, השרת שמבצע את העבודה, ומסד הנתונים שבו כל הפניות שמורות. המסך לא שומר את כל הנתונים אצלו – הוא תמיד שואל את השרת, והשרת שולף בדיוק את מה שצריך.

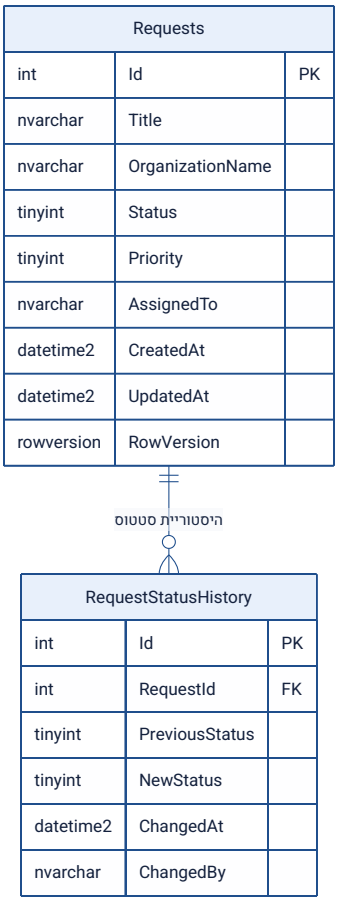


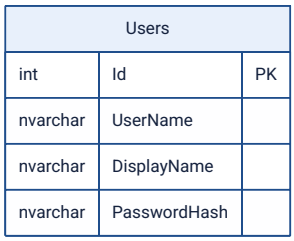
אפליקציית Angular פונה ל־REST API עם טוקן JWT, וה־API ניגש ל־SQL Server דרך EF Core; כל הסינון, המיון והדפדוף מתבצעים ב־SQL.

3 מודל הנתונים ומעברי הסטטוס

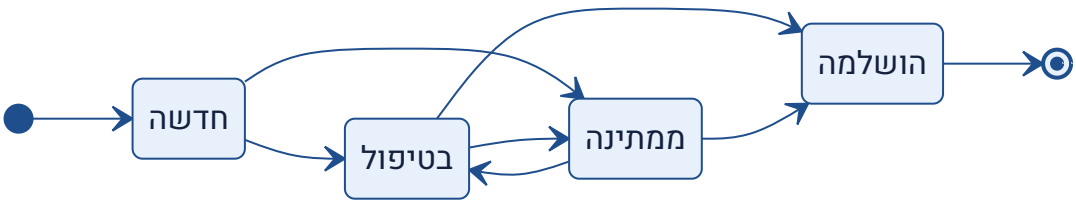
במילים פשוטות

לכל פנייה נשמרים פרטים כמו כותרת, ארגון, סטטוס ומי מטפל בה. כל שינוי סטטוס נרשם ביומן נפרד, כך שתמיד אפשר לראות את ההיסטוריה. בנוסף, פנייה יכולה לעבור רק בין מצבים מסוימים – למשל, פנייה שהושלמה לא חוזרת לטיפול.





`Users` הם משתמשי הדמו להתחברות. `AssignedTo` ו־`ChangedBy` שומרים שם משתמש כטקסט, בלי מפתח זר. `RowVersion` הוא הבסיס ל־Concurrency. האינדקסים והנימוק לכל אחד – ב־PERFORMANCE.md.

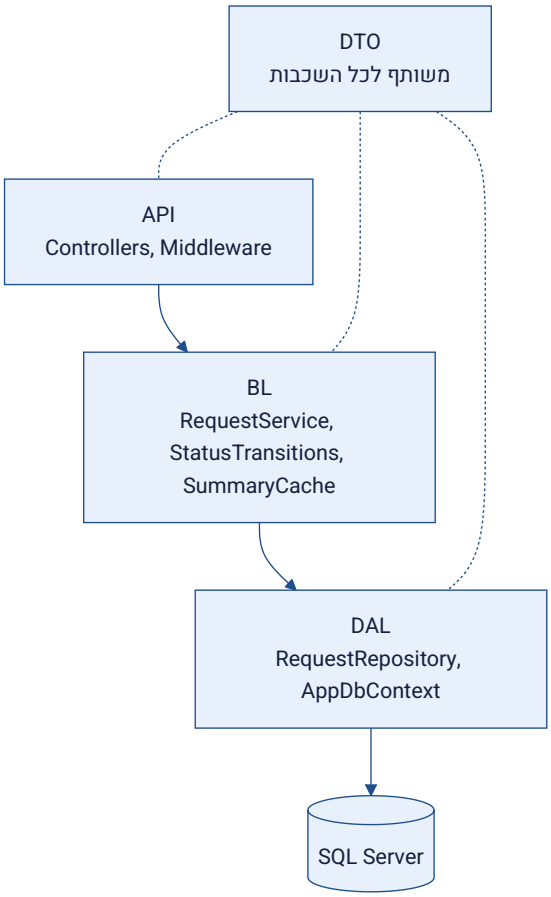


"הושלמה" הוא מצב סופי, ומעבר לאותו סטטוס חסום (D-05). מעבר אסור מחזיר `422`. המטריצה מוגדרת במקום אחד בשרת (`StatusTransitions`), ובלקוח יש עותק לתצוגה בלבד (D-35).

4 השרת

במילים פשוטות

השרת בנוי בשכבות, כמו קומות בבניין: הקומה העליונה מקבלת בקשות, האמצעית מחליטה מה מותר ומה לעשות, והתחתונה מדברת עם מסד הנתונים. כך כל חלק אחראי על דבר אחד, וקל לבדוק ולתחזק אותו.



ה־Entities הם internal ב־DAL, והשאילתות מחזירות DTOs ישירות (D-03, D-04).

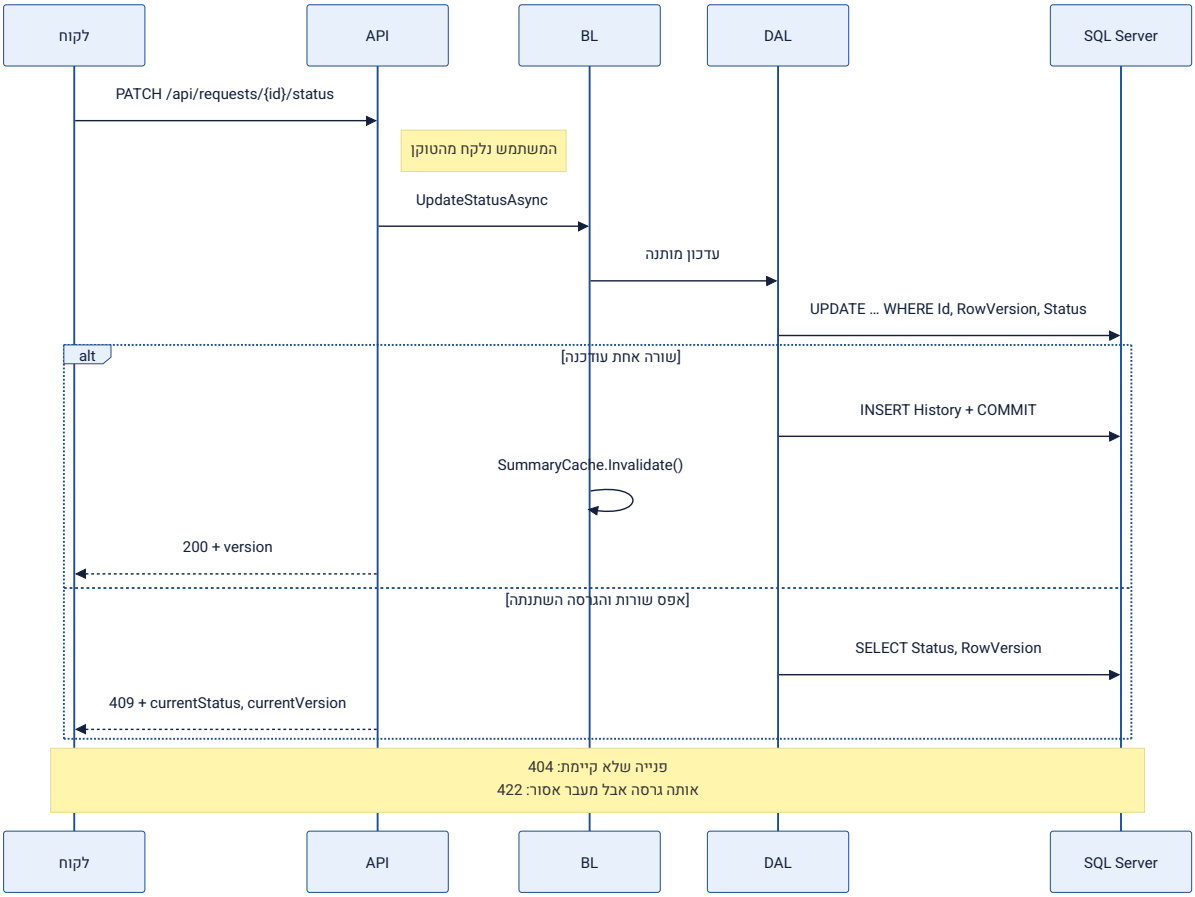
הרכיבים העיקריים

`RequestsController`, `AuthController`	ה־Endpoints; פונים רק לממשק של ה־BL.
`GlobalExceptionHandler`	כל שגיאה חוזרת כ־ProblemDetails עם `traceId`; ניתוק לקוח מסתיים ב־499 בלי שגיאה ביומן.
`ValidationProblemResponse`	תשובות 400 נקיות: שמות שדות פשוטים (`newStatus`, `items[0].id`) והודעות קריאות, בלי שמות טיפוסים פנימיים של .NET (D-18).
`RequestService`	לוגיקת העסק; מתודה אחת לשינוי סטטוס, משותפת לעדכון הבודד ול־Bulk.

`StatusTransitions`	מטריצת המעברים, במקום אחד.
`SummaryCache`	ה־Cache של הסיכום: Singleton, 60 שניות ומונה "דור" (D-20).
`RequestRepository`	שאילתות עם Projection ל־DTO, דפדוף יעיל ב־Deferred Join (דילוג על עמודים באינדקס צר, ושליפת שורות העמוד בלבד), ועדכון סטטוס מותנה עם `OUTPUT`.
`DatabaseSeeder`	100,000 פניות דטרמיניסטיות ב־SqlBulkCopy (D-13, D-33).

עדכון סטטוס, כולל עדכון מתחרה (409)

> **במילים פשוטות**
>
> נניח ששני אנשים פתחו את אותה פנייה ולוחצים "עדכן" כמעט באותו רגע. רק הראשון מצליח. השני מקבל הודעה שהפנייה השתנתה בינתיים, ורואה את המצב המעודכן – כך אף שינוי לא נמחק בשקט.

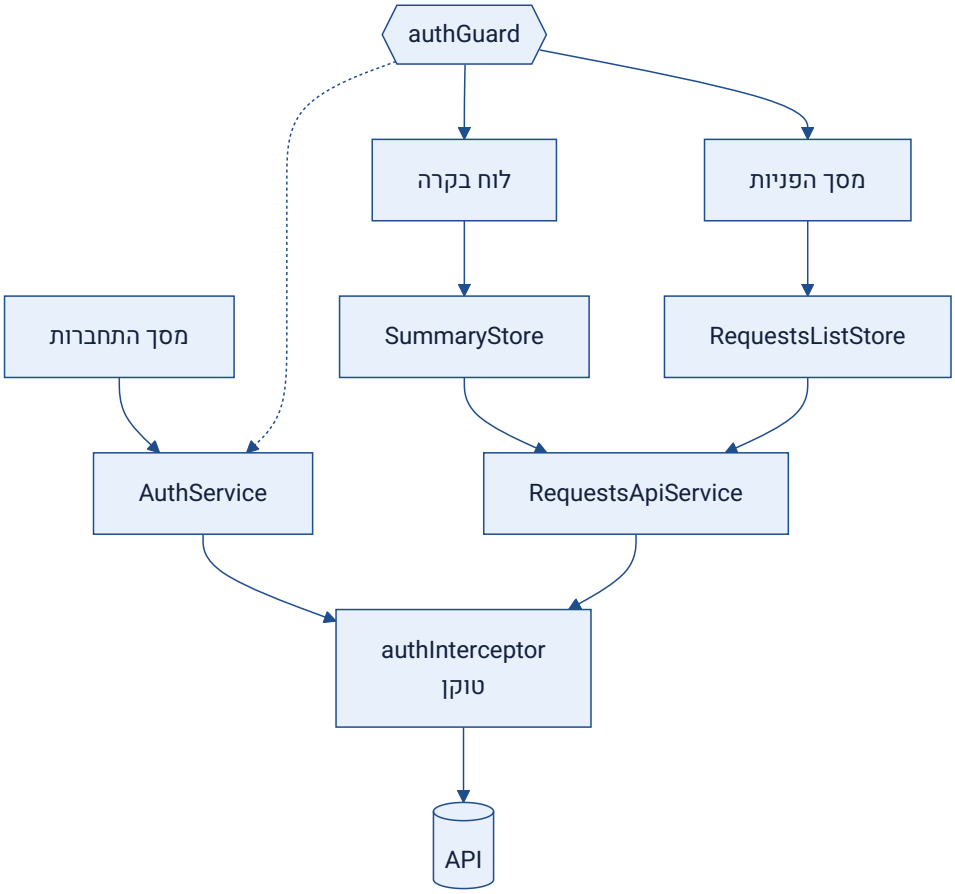


העדכון ושורת ההיסטוריה נכתבים באותה טרנזקציה, וה־DB מבטיח שרק אחד מכמה עדכונים מקבילים עם אותה גרסה יצליח (D-06, D-07). חוזה ה־API המלא – ב־API.md.

5 הלקוח

במילים פשוטות

לממשק יש שלושה מסכים: התחברות, לוח בקרה עם סיכום מספרי, ומסך הפניות עם טבלה, סינון וחיפוש. "שומר בכניסה" מוודא שרק משתמש מחובר רואה את המסכים, ושירותים ברקע מטפלים בשיחה עם השרת.



שירותי הליבה

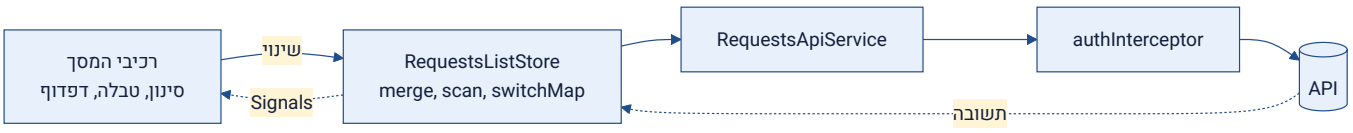
`AuthService`	ההתחברות והסשן (ב־`sessionStorage`), ובדיקת תוקף במקום אחד (D-26).
`authGuard`	מגן על לוח הבקרה ועל מסך הפניות; משתמש לא מחובר מועבר להתחברות.
`authInterceptor`	מוסיף את הטוקן לקריאות ה־API, ו־401 מוביל להתחברות מחדש.
`RequestsApiService`	כל הקריאות ל־API של הפניות.
`SummaryStore`, `RequestsListStore`	מצב המסך; כל אחד מסופק ע"י המסך שלו ונזרק ביציאה ממנו.

הרכיבים העיקריים

`LoginComponent`	טופס ההתחברות.
`DashboardPageComponent`, `SummaryCardsComponent`	לוח הבקרה: כרטיסי סיכום וכפתור "רענן".
`RequestsPageComponent`	מסך הפניות.
`RequestsFiltersComponent`	סינון וחיפוש.
`RequestsTableComponent`	הטבלה, מיון ודפדוף, ותפריט שינוי סטטוס לכל שורה.
`BulkBarComponent`, `BulkResultDialogComponent`	בחירה מרובה ותוצאה לכל פנייה.
`ConfirmCompleteDialogComponent`	אישור לפני מעבר ל"הושלמה".
`RequestHistoryDialogComponent`	היסטוריית הסטטוס של פנייה.
`StateMessageComponent`	הודעות טעינה, ריק ושגיאה.

זרימת הנתונים במסך הפניות

> **במילים פשוטות**
>
> כשמקלידים בחיפוש, המערכת מחכה רגע קצר עד שמסיימים להקליד, ורק אז שואלת את השרת. אם שינית את החיפוש לפני שהתשובה הגיעה, הבקשה הישנה מבוטלת – כך תמיד רואים את התוצאה של החיפוש האחרון.



כל שינוי – עמוד, מיון, מסנן או טקסט (אחרי debounce של 300 ms) – מתקפל לשאילתה אחת בזרם RxJS יחיד; `switchMap` מבטל בקשה קודמת שעוד לא חזרה, והתוצאה מגיעה לרכיבים כ־Signals (D-27, D-28).

6 החלטות מרכזיות

במילים פשוטות

בכל צומת בפרויקט נשקלו כמה דרכים, ונבחרה אחת – עם הסבר למה. כאן מופיעות 12 ההחלטות החשובות. כל 43 ההחלטות, עם החלופות שנשקלו, נמצאות ב־DECISIONS.md.

כל 43 ההחלטות הפעילות, עם הסיבה והחלופות שנשקלו – ב־DECISIONS.md. העיקריות:

D-01	**SQL Server:** `rowversion` מובנה ל־Optimistic Concurrency, טרנזקציות ו־Execution Plans ברורים.
D-03	**ארבעה פרויקטים בשרת (API, BL, DAL, DTO):** הפרדה מוכרת וברורה, בלי שכבות שרק מעבירות קריאות.
D-06	** `UPDATE` מותנה, ו־409 לעדכון מתחרה:** האכיפה ב־DB, אטומית, ומונעת Lost Update.
D-08	**Bulk עם Partial Success:** כל פנייה עצמאית, וכשל באחת לא מבטל את השאר.
D-10	**JWT, ו־ `ChangedBy` מתוך הטוקן:** השרת קובע מי שינה, והלקוח לא יכול לזייף.
D-49	**Repository אחד:** קישור אחד לבודק, עם ההיסטוריה המלאה של ה־Backend וה־Frontend ו־CI נפרד לכל תיקייה.
D-20	**Cache עם מונה "דור" ו־Invalidation אחרי ה־Commit:** קורא איטי לא משאיר נתון ישן אחרי שינוי.
D-21	**בדיקות מול SQL Server אמיתי:** הערבויות המרכזיות (`rowversion` , טרנזקציה, אינדקסים) הן של ה־DB.
D-24	**דפדוף בשרת ואינדקסים שנבחרו במדידה:** כל אינדקס נבדק ב־logical reads, ב־Execution Plan ובעלות הכתיבה.
D-27	**זרם RxJS יחיד עם `switchMap` :** בקשות מיושנות מתבטלות, וכל הסינון נשאר בשרת.
D-43	**לוח הבקרה כמסך הפתיחה:** סקירה קודם, והרשימה במרחק לחיצה.
D-18	**Validation אחד:** DataAnnotations על ה־DTOs, וכל קלט לא תקין – 400 באותו פורמט ProblemDetails, עם שמות שדות והודעות קריאות.

7 מגבלות ידועות ובחירות מודעות

במילים פשוטות

כל פתרון הוא איזון בין פשטות ליכולת. כאן ארבעה דברים שנבחרו במודע להשאיר פשוטים בהיקף הזה: מה המחיר של כל אחד, ומה היה נדרש במערכת אמיתית.

ארבע מגבלות שנבחרו במודע בגלל היקף המבדק. הרשימה המלאה – בסוף DECISIONS.md.

חיפוש טקסט חופשי

`LIKE '%x%'` אינו SARGable: אינדקס לא עוזר, והחיפוש סורק את כל הטבלה (כ־52 ms ב־000,100 פניות, וגדל ליניארית). בקנה מידה גדול: אינדקס n-grams או Full-Text (D-12, D-24, PERFORMANCE.md §7.1).

במילים פשוטות: חיפוש חופשי בודק כל שורה בטבלה, כמו לקרוא ספר שלם כדי למצוא מילה. ב־000,100 פניות זה מהיר; במיליון פניות היינו בונים אינדקס חיפוש ייעודי.

Cache במופע שרת יחיד

`IMemoryCache` חי בזיכרון של תהליך אחד; בכמה מופעים עלול להישאר summary ישן עד 60 שניות. הפתרונות (Redis, HybridCache, Pub/Sub) מתועדים ב־CACHE.md ולא מומשו (D-11, D-20).

במילים פשוטות: הזיכרון הזמני שייך לשרת אחד. אילו היו כמה שרתים, כל אחד היה מחזיק עותק משלו, וצריך מקום משותף שיסנכרן ביניהם.

Bulk רציף עם Partial Success

הפריטים מעובדים אחד אחרי השני, כל אחד בטרנזקציה משלו ובאותה לוגיקה כמו PATCH בודד: אין Deadlock וכשל באחד לא מבטל את השאר. המחיר: הזמן גדל עם מספר הפריטים (כ־0.2 שניות ל־100) (D-08, D-19).

במילים פשוטות: הפניות בעדכון המרוכז מטופלות בתור, אחת אחרי השנייה. זה בטוח ופשוט, במחיר של קצת יותר זמן כשיש הרבה פניות.

אימות ברמת דמו

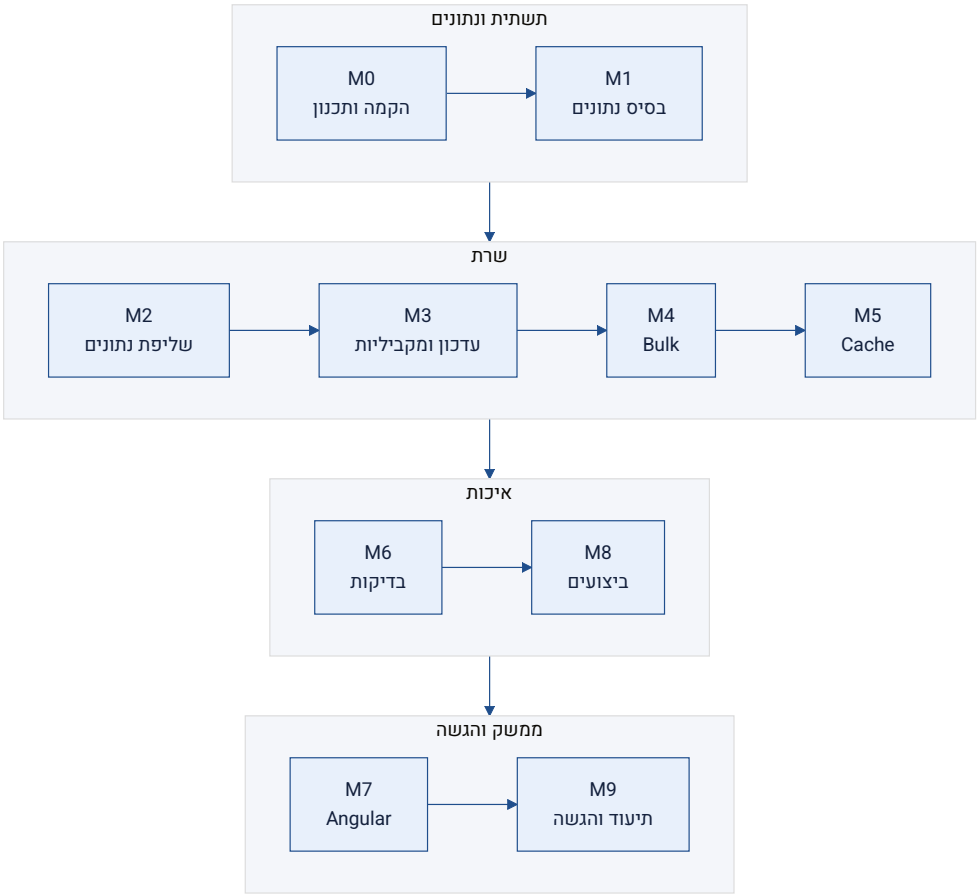
סיסמת דמו אחת, בלי הגבלת ניסיונות, וטוקן JWT תקף 480 דקות (8 שעות) ואינו ניתן לביטול בשרת. התנתקות מכל הלשוניות מטופלת בצד הלקוח (בשינוי נפרד בעבודה); ביטול טוקנים בשרת מחוץ להיקף. בייצור נדרשים סיסמאות אישיות, Rate Limiting, Refresh Tokens ו־Secret Store (D-10).

במילים פשוטות: ההתחברות כאן מיועדת להדגמה: משתמשי דמו וסיסמה אחת. מערכת אמיתית הייתה צריכה ניהול משתמשים מלא ואבטחה חזקה יותר.

8 תהליך העבודה

במילים פשוטות

העבודה חולקה לעשרה שלבים, וכל שלב הסתיים במשהו שעובד ונבדק. לפני כל שינוי הוצגה תוכנית ואושרה, ואחרי כל שינוי הוא נבדק – אוטומטית ובדפדפן – ונשמר עם תיעוד.



עשרה Milestones בסדר שבו בוצעו; M8 (ביצועים) בוצע לפני M7 (Angular), כדי לסיים את השרת לפני הממשק (D-22). התלויות, המשימות והערכת המאמץ – ב־WORK_PLAN.md.

הערכה מול בפועל: 61.5 שעות בהערכה; לפי דיווח עצמי העבודה ארכה הרבה פחות, כי כלי ה־AI קיצרו את כתיבת הקוד (WORK_PLAN.md).

מחזור העבודה בכל משימה: תוכנית ← אישור מפורש ← ביצוע ← אימות ← Commit עם הפניה ל־Issue, Push ו־CI ירוק.

שימוש ב־AI: Claude Code – Opus לתכנון, ארכיטקטורה וסקירה, ו־Sonnet לכתיבת קוד ובדיקות. כל קוד שנכתב אומת בשיטות שלמטה. הפירוט – ב־AI_USAGE.md.

שיטות האימות

Oracle מול SQL	בזמן הפיתוח סקריפט אימות השווה סינון, חיפוש, מיון ודפדוף לשאילתות SQL ישירות על כל 000,100 הפניות; הוא הוסר בניקוי האחרון (#54), ומקור האמת הן הבדיקות האוטומטיות.
Oracle ב־LINQ	בבדיקות ה־Integration התוצאה הצפויה מחושבת מהנתונים ולא מועתקת מהקוד (D-21).
בדיקות מוטציה	שוברים את הקוד בכוונה ומוודאים שהבדיקות נכשלות.
הוכחת Concurrency	`ConcurrencyTests` : 10 פניות × 20 בקשות PATCH בו־זמנית עם אותה גרסה ← לכל פנייה בדיוק `200` אחד ו־19 תשובות `409` , ושורת היסטוריה אחת; ו־Bulk במקביל ל־PATCH ← מנצח אחד לכל פנייה, בלי 5xx. העדכון ושורת ה־Audit אטומיים (`UpdateAtomicityTests`).
דפדפן אמיתי	ב־Chrome מול ה־API האמיתי, לפני ואחרי כל תיקון בממשק: יישור טקסט מעורב עברית/אנגלית, ניווט ומיקוד מקלדת, וזמן הבקשה הראשונה לסיכום (78–105 ms אחרי החימום).
בדיקה מקצה לקצה	מ־clone נקי על LocalDB, כמו בודק; התאריך והתוצאות – ב־README.

9 מבנה התיקיות

במילים פשוטות

שלושה מאגרים: אחד לתיעוד, אחד לשרת ואחד לממשק. בכל אחד יש סדר קבוע – קוד, בדיקות ותיעוד – כך שקל למצוא כל דבר.

	`requests-management/`
נקודת הכניסה לבודקים	`README.md`
מסמך זה	`PROJECT_SUMMARY.md`
CI: be.yml (build ובדיקות מול SQL Server בקונטיינר), fe.yml (lint, בדיקות ו־build)	`.github/workflows/`
החלטות, API, ביצועים, Cache, תוכנית עבודה, שימוש ב־AI	`docs/`
תוצאות המדידות ו־Execution Plans: before, after-indexes, final, alternatives, seed-v2-hebrew	`performance/`
Backend	`be/`
	`RequestsManagement.sln, global.json`
מדידת שאילתות (MeasureQueries.cs)	`scripts/`
	`src/`
Controllers, Middleware, Auth, Startup, Program.cs	`RequestsManagement.API/`
RequestService, StatusTransitions, SummaryCache, AuthService	`RequestsManagement.BL/`
Entities, Configurations, Migrations, Seeding, RequestRepository	`RequestsManagement.DAL/`
מודלי בקשה ותשובה, Enums	`RequestsManagement.DTO/`
	`tests/RequestsManagement.Tests/`
בדיקות יחידה	`Unit/`
בדיקות דרך ה־API מול SQL Server, ותשתית הבדיקות	`Integration/`
Frontend	`fe/`
	`angular.json, package.json, proxy.conf.json`
	`src/app/`
API, התחברות, עברית, אייקונים, ניתוב	`core/`
המסכים: login, dashboard, requests	`features/`
רכיבי תצוגה משותפים: תגי סטטוס ועדיפות, הודעות מצב	`shared/`

10 מפת המסמכים

במילים פשוטות

המסמך הזה הוא סקירה. לכל נושא יש מסמך מפורט – הטבלה מראה לאן להמשיך.

נושא	מסמך
הרצה, בדיקות ומספריהן, ופריטי התיעוד של סעיף 11	README.md
שימוש בכלי AI	docs/AI_USAGE.md
כל ההחלטות, עם סיבה וחלופות, ומגבלות ידועות	docs/DECISIONS.md
חוזה ה־API, Bulk וקודי שגיאה	docs/API.md
אינדקסים, Execution Plans ו־Bottleneck	docs/PERFORMANCE.md
ה־Cache, ה־Invalidation וריבוי מופעים	docs/CACHE.md
Milestones, משימות, הערכות ומיפוי בדיקות לדרישות	docs/WORK_PLAN.md
תוצאות המדידות הגולמיות ומדריך לתיקיות	docs/PERFORMANCE.md – נספח ב'
הבדיקה מקצה לקצה מ־clone נקי	README.md – בדיקות
הרצת ה־Backend ומדידת השאילתות	be/README.md
הרצת ה־Frontend	fe/README.md

11 מילון מונחים

למי שאינו מפתח

הסבר קצר למונחים הטכניים שמופיעים במסמך.

API	"שפת הדיבור" של השרת: הכתובות שהממשק פונה אליהן כדי לקבל או לשנות נתונים.
Backend / Frontend	השרת, שמבצע את העבודה מאחורי הקלעים / הממשק שהמשתמש רואה בדפדפן.
Repo (מאגר)	תיקיית קוד מנוהלת ב־Git, עם כל ההיסטוריה של השינויים.
מסד נתונים / SQL	המקום שבו הנתונים שמורים, ו־SQL היא השפה לשליפה ולעדכון שלהם.
אינדקס	כמו אינדקס בסוף ספר: מאפשר למסד הנתונים למצוא שורות מהר בלי לעבור על כולן.
SARGable	שאילתה שמסד הנתונים יכול להאיץ באינדקס. חיפוש "מכיל" באמצע הטקסט אינו כזה, ולכן עובר על כל השורות.
Execution Plan	"מפת הדרכים" שמסד הנתונים בוחר כדי לבצע שאילתה; משמשת לבדיקת ביצועים.
Deferred Join	דפדוף חכם: קודם מוצאים באינדקס צר אילו שורות שייכות לעמוד, ורק אז שולפים את הפרטים המלאים שלהן – במקום לשלוף ולזרוק את שורות העמודים הקודמים.
Cache	זיכרון ביניים: תוצאה ששמורה לזמן קצר כדי לא לחשב אותה שוב בכל פעם.
Concurrency	עבודה של כמה אנשים במקביל על אותם נתונים, בלי שאחד ימחק בטעות את השינוי של השני.
409	קוד תשובה שאומר "מישהו אחר שינה את הפנייה בינתיים" – המשתמש מקבל את המצב המעודכן.
400	קוד תשובה שאומר "הבקשה לא תקינה". התשובה מציינת את שם השדה ומסבירה מה לא בסדר, בלי פרטים פנימיים של השרת.
Bulk	פעולה על כמה פריטים יחד, למשל שינוי סטטוס לעשרות פניות בלחיצה אחת.
Partial Success	בפעולה מרוכזת, מה שהצליח נשמר ומה שנכשל מדווח בנפרד – כשל באחד לא מבטל את השאר.
JWT / טוקן	"כרטיס כניסה" דיגיטלי שמקבלים בהתחברות ומצרפים לכל בקשה, כדי שהשרת ידע מי המשתמש.

Seed	יצירה אוטומטית של נתוני דוגמה, כדי שיהיה על מה לעבוד ולבדוק.
CI	בנייה ובדיקות שרצות אוטומטית ב־GitHub אחרי כל שמירה של קוד.
Milestone	שלב בתוכנית העבודה, שמסתיים בתוצר שעובד ונבדק.
debounce	המתנה קצרה עד שהמשתמש מסיים להקליד, לפני ששולחים בקשה לשרת.
בדיקת מוטציה	שבירה מכוונת של הקוד כדי לוודא שהבדיקות באמת מזהות תקלות.